

Generalization as Formalized Induction: Token-Native Factorized Representation and Weight Compilation

> **Draft v0.1 (English)** | 2026-08-11
> Target: NeurIPS/ICML/ICLR (theory + empirical)
> Working notes: `P1\_0000000000.md` (theory skeleton), `P1\_00.md` (Chinese outline)
> Evidence: `docs/paper\_data/` (all experiments via run_exp official judge evaluation)

Abstract

We propose that systematic generalization is not an emergent or a priori faculty of neural networks, but the formalization of induction. To generalize on a concept A, A must be precisely defined, and every concept in A's definition chain must be sufficiently trained; sufficient training is a relational property – simultaneous training of parallel concepts under the same superordinate concept, wrong-answer examples, and collocation coverage – not a quantitative one. We operationalize this in a token-native representation (six projection layers: axioms B, concepts C, symbols S, grammar G, presentation P, arrows A) in which conceptual degrees of freedom are factorized according to their symmetries, and execution is driven by token definitions (a zero-hardcoding evaluator). A 2-layer, <1 MB transformer generalizes to arbitrary width (2000-digit addition, acc 1.000), unseen radices ([3..9], OOD 0.998), unseen domains, and joint Cartesian OOD – with ~2000 samples/3 epochs for a simple arithmetic suite and ~6000 samples/8 epochs for the full logic-and-arithmetic suite. Symbol permutation invariance holds (0.996-0.987 under 26080 digit renamings), and the three execution channels – construction, intuition, formal rewriting – are semantically equivalent (1.000 on 2000-digit carry chains), constituting the neural implementation of Computational Trinitarianism. We emphasize that intuition is the reasoning fast path: it is hard to obtain (acquisition requires config-dependent epochs – 3 for a simple suite, 8 for the full logic-and-arithmetic suite), but once formed it generalizes easily by skipping steps, jumping from input to output over the intermediate construction steps, with zero decay out to 2000-digit width, radix 60, and 100% digit anonymization. Acquisition and step-skipping generalization are decoupled. Per-concept learning curves are sharply bimodal – a concept is either compiled (100% by epoch 3) or not (0%, unfixed by more epochs) – the mechanism-level signature that generalization is the formalization of induction rather than a matter of training volume.

1 Introduction

****The problem.**** Compositional generalization (Lake et al., 2017; Keysers et al., 2020) is a well-documented phenomenon: models sometimes respond correctly to combinations absent from training. Yet the literature asks **whether** models generalize, not **how** generalization arises. Three defaults dominate: (i) generalization is an emergent/a priori faculty; (ii) generalization follows from data engineering at scale; (iii) the relevant structure must be provided by an external program (neural-symbolic hybrids, program synthesis). Each default precommits to a mechanism without explaining the observed pattern. In the philosophy of mathematics this is the old dispute resurfaced: Platonism (structure exists a priori, to be discovered), intuitionism (existence = constructibility, Brouwer), and formalism (mathematics = sign games, Hilbert) – each precommits a source for structure.

Meanwhile, the dominant approach – “existing mathematical language, then tokenize, then train” – pre-supposes primitives. λ -calculus fixes variable/abstraction/application (Barendregt 1984); Church encoding **implements** any concept but erases all structural boundaries in the representation – a model trained on the encoded surface can memorize the function table but has no access to the structure that would make out-of-distribution combinations easy. A concrete instance is the “neural λ -calculus” program (Flach, Moreira & Lamb 2023): a seq2seq Transformer is trained on standard λ -syntax with de Bruijn indices to perform β -reduction, reaching 97.7% on Boolean terms – but generalization is confined to the λ -term distribution it was trained on; there is no width/radix/order extrapolation, no symbol permutation, no concept formation. This is precisely the “ λ -tokenize-then-train” route this paper argues is incomplete. Our internal structural analysis of λ -iteration ($f^{m+n} = f^m \circ f^n$; the Church zero is not a base; iteration becomes a universal property under initiality) shows that the relevant structure is genuinely present in mathematics, but is **erased** by the encoding – it must be re-exposed in the representation for a learner to use it.

****What we do.**** We take the opposite route. We treat systematic generalization as an engineering target: it is produced, not assumed. The producing mechanism is the formalization of induction – a concept A generalizes exactly when (i) A is precisely defined and (ii) every concept in A's definition chain is sufficiently trained, where sufficiency is relational (parallel concepts, wrong answers, collocations). We build a token-native representation in which these conditions are verifiable, and show that a minimal transformer then generalizes essentially for free.

****Contributions**** (each a falsifiable claim; the propositions G0, G1, G2, G4, G5 and R are stated in §3):

- ****C1 (Theory) – Generalization as formalized induction.**** Systematic generalization is not emergent/a priori: definition completeness (G1) and relational sufficiency (G2) are sufficient. The anti-prior claim is empirically falsifiable (definitionally-impovertised tokens do not generalize even with abundant data).
- ****C2 (System) – Token-native factorized representation.**** Six projection layers (B/C/S/G/P/A) with degrees of freedom factorized by symmetry, and a zero-hardcoding evaluator; “definition precision” becomes checkable (no dangling references, cycle certificates, coverage audits).
- ****C3 (Empirics) – Minimal-model systematic generalization.**** A 2-layer, <1 MB transformer: arbitrary width (2000-digit, 1.000), unseen radices (OOD 0.998), unseen domains, joint OOD; symbol-permutation invariance (0.987); three-channel semantic equivalence (1.000). Sample counts: ~2000 (simple) / ~6000 (full suite); epochs 3 (simple) / 8 (full).
- ****C4 (Mechanism) – Weight compilation.**** Training compiles the definition-driven construction channel into weights; supervision excludes answer traces, so the model learns structure rather than replicating the unfolding. The product is **intuition* – the reasoning fast path: hard to obtain, easy to generalize by step-skipping.
- ****C5 (Epistemology) – Three-channel equivalence.**** Construction / intuition / formal rewriting are pairwise semantically equivalent on the same inputs – the neural implementation of Computational Trinitarianism (equivalence between execution channels of one system, not between logics).

2 Problem Setting

****Systematic generalization.**** The ability to respond correctly to combinations not present in training, where combinations span independent degrees of freedom: width $\times$ radix $\times$ operator order $\times$ domain $\times$ nesting depth. We test extrapolation in these dimensions, never interpolation.

****In-distribution vs OOD.**** A degree-of-freedom value is in-distribution if its combination appears in training; OOD otherwise. Single-dimension extrapolation (width beyond training length; radix absent from training; order beyond the highest trained) and joint extrapolation (unseen points of the Cartesian product) are both tested. Primary metric: ***judgment accuracy*** – full output-sequence reconstruction of the final truth token (the only trusted metric; positional accuracy can mask structural errors, e.g. a 0.904 positional score with 0.000 judgment accuracy on a focused-configuration artifact). All evaluation uses the single official run_exp judge path.

****Notation and conventions.**** A ***concept*** (token) is the unit of meaning; a ***definition*** is a structured statement of a concept's content, among five forms (explicit, inductive, recursive, axiomatic, implicit). A ***definition chain*** of A is the transitive closure of concepts that A's definition references. ***Sufficient training*** of a concept means relational coverage: it participates with its parallel concepts, its negative examples, and its collocations. ***Judgment*** is a complete output sequence (expression, result, truth); ***judgment accuracy*** is the fraction of full reconstructions correct.

****Grammar.**** Expression grammar is data-driven (G-layer gtoken arrangement + P-layer presentation), with slots arg/fn/args/binder/body and node types atom/application/equality/binary_connective/unary_connective/quantified/eos. Numerals are a sign/radix/cardinality intrinsic vector combined with a digit sequence; any radix is representable (no single-character names needed). Propositions are judged along gtoken/ptoken-assembled sequences.

3 Theory: Formalized Induction

We state the theory as six falsifiable propositions (G0, G1, G2, G4, G5 and R). Each is a claim about when and how generalization is produced; each is supported by experiments in §6 and ablations in §8.

****G0 (epistemic ground) – Generalization is the formalization of induction.**** ***There is no a priori or transcendental generation of conceptual ability: any ability to generalize on a concept must be produced, not assumed.*** Empirically falsifiable: if concept ability were a priori, definitionally-impoorished concepts would still generalize; they do not – an impoverished vocabulary requires an order of magnitude more training to converge (40 vs 3 epochs, §8), and genuinely under-defined tokens never generalize. Grounding: this is the anti-Platonist / anti-nativist position (Brouwer's constructivism in machine form; Fodor 1975; Chomsky 1959 as the opposing positions).

****G1 (definition completeness).**** ***To generalize on concept A, (i) A's definition must be precise and complete, and (ii) every concept in A's definition chain must itself be sufficiently trained. Both conditions are necessary; neither suffices alone.*** Evidence: a definition-chain token with a single training instance (value_three) is not learned until coverage is added; a definitionally-impoorished vocabulary requires 40 epochs vs 3 for a precise one (§8). Definition forms follow Aczel 1977, Gupta 2023, Belnap 1993.

****G2 (relational sufficiency).**** ***Sufficient training is a relational property, not a quantitative one: (i) simultaneous training of parallel concepts under the same superordinate concept makes relations between concepts explicit learning signals; (ii) wrong-answer training (negative examples, including structure-identical-result-wrong) defines what a concept is not; (iii) collocation training covers each participating token's adjacent collocations above a threshold.*** Evidence: removing one gate-family's supervision collapses that gate to 0.000 and ***degrades all other gates from 1.0 to 0.833*** (§8); increasing negative-example coverage lifts a relation from 0.000 to 0.958.

****G4 (modal separation).**** ***Mirroring perception–meaning–expression, model input, logic, and output are assigned independent grammars; signifier and signified are separated at the token level.*** Evidence: assigning one symbol to two semantic wrappers collapses training (§8). Grounded in semiotics (Saussure 1916; Frege 1892) and in abstract/concrete syntax separation.

****G5 (intuition/reasoning separation).**** ***Intuition is the reasoning fast path: hard to obtain (acquisition requires config-dependent epochs), but easy to generalize by skipping steps – it jumps from input to output over the intermediate construction steps.*** Evidence: acquisition needs 8 epochs for the full suite (exp53) while step-skipping generalization shows zero decay at 2000 digits, radix 60, and 100% anonymization (exp80) – acquisition and step-skipping generalization are decoupled (§6).

****R (three-channel equivalence).**** ***A single system exhibits three execution channels for the same mathematical object – the construction channel (unfolded, definition-driven expansion), the intuition channel (compiled weight execution), and the formal channel (definition-driven rewriting) – and they are pairwise semantically equivalent: construction can be compiled into intuition, and both agree with formal rewriting.*** This is the neural implementation of Computational Trinitarianism (Curry–Howard: programs = proofs = types), with equivalence holding between execution channels of one system rather than between logics. Evidence: at a 2000-digit carry chain, construction = formal exactly and intuition scores 1.000 (§6).

4 The Representation System

****Six projection layers.**** A token is one object; the six JSON projections are projections of it.

Layer	Entity	Role
---	---	---
B	axioms	baseloop axiomatic endpoints; terminal of definition-chain tracing
C	concepts	derive five definition forms (explicit/inductive/recursive/axiomatic/implicit)
S	symbols	symbol signifiers (glyph → maps-to-many; ambiguity preserved, resolved by context)
G	grammar	gtoken arrangement methods (slots/reduction); syntax itself is tokens
P	presentation	presentation concrete notation (symbol/precedence/associativity); one concept, many notations
A	arrows	arrow directed relations between concepts (domain lifts, iteration chains, duals)

Transformations depend only on tokens; knowledge lives in token/grammar data; code reads and executes. This is the zero-hardcoding discipline. The six-layer projection is a formalization of representation rather than a new logic: it separates signifier from signified (Saussure 1916; Frege 1892), makes arrangement a definable object (van Wijngaarden 1965; Ford 2004; Rendel & Ostermann 2010), separates binding structure from syntax (de Bruijn 1972; Fiore–Plotkin–Turi 1999; Gabbay & Pitts 2002), and encodes concept relations as first-class arrows (Goguen et al. 1977; Mac Lane 1971).

****Degrees of freedom.**** A concept is a coordinate (d1,...,dk), not an isolated token: iteration order (succ→+→x→^→↑↑),

duality/direction (inverse: succ/pred, power/root, +/-), domain (natural/integer/rational/real/complex), and notation (sign/base/cardinality).

****Symmetry.**** Iterative symmetry: $\text{iterate}(\text{direction}, \text{level}, \text{operator})$ is a higher-order operator; $F_{n+1}(a,b) = \text{Iterate}(F_n, a, b)$. Structurally, iteration is composition ($f^{m+n} = f^m \circ f^n$): it yields a monoid, then a category, then a functor and natural transformation under semiconjugacy – the standard route from λ -iteration to category theory (Lambek 1968; Mac Lane 1971; our λ -base iteration study). Dual symmetry: inverse arrows (power $\rightarrow$ root), symmetry families (neg/scale/root; complement/parallel_sum as De Morgan duals; translation/inversion as modular group). Domain lifts: coercion as arrows (retrieved by `concept=coercion`, zero hardcoding), in the style of canonical inclusions between algebraic structures (Mac Lane 1971; Adámek et al. 2004).

****Composition and iteration.**** Arrangement lives in the G-layer; assembly reads data only; SKI combinators are gtokens (reduction rules in data). Iteration is an operator over operators; iteration-chain membership is discovered along A-layer arrows (ARROW_REGISTRY), never via name hardcoding. The generic evaluator handles pure chain members by iterative reduction and special-symmetry members (translation/inversion/differential) by delegated symmetric evaluation – token in, token out. This mirrors the definitional structure of iteration (operational $\rightarrow$ universal property under initiality; Lambek 1968; Goguen et al. 1977), kept as data rather than code.

****Domain/radix representation.**** The radix is an *object*, not a structural constant: a numeral is [sign, base, cardinality] $\times$ digit vector; one set of weights handles any base (base as input parameter). This follows the standard analysis of positional notation (base as a parameter of representation, not of the number – see our number-system survey); notations are extensible (fraction/radical/imaginary_unit) while numeral token sequences remain digit-arranged (never "directly by value"). Different semantic wrappers require different symbols – operation nesting "(...)" vs representation nesting "[...]" – a modal separation whose violation collapses training (see §8); this echoes the parse/render separation between concrete and abstract syntax (Rendel & Ostermann 2010; Fiore-Plotkin-Turi 1999).

5 Weight Compilation

****Data generation.**** Samples are instantiations of definitions. Concepts carry one of five definition forms (explicit, inductive, recursive, axiomatic, implicit), following the standard theory of definitions (Aczel 1977; Gupta 2023; Belnap 1993; Suppes 1957): inductive definitions rest on least-closure, implicit definitions on conservativeness and eliminability. The synthesizer (config-driven) composes: balanced / trace / numeral_split / fill / choose / definition / arrow / law / logic_nested / logic_arith / logic_interdef / coercion / deep_nest / extrap / cartesian / radix / neg. Sufficiency (G2) requires: parallel concepts trained together (relations become learning signals), wrong answers (negative examples, including structure-identical-result-wrong), and collocation coverage (adjacent-pair coverage ≥ 3). Sequences are never hand-written; assembly follows gtoken/ptoken (hand-written sequences are flagged as "missing-token" signals).

****Reference evaluator.**** The generic evaluator computes from token definitions: logic/compare truth tables, arithmetic iteration chains, symmetry families, primality – all read from definitions. Construction channel (token_iterator) and formal channel (rewriting engine) share the same definition source, which is the basis of three-channel consistency.

****Training.**** A 2-layer, dim-64 transformer (<1 MB). Two objectives: positional cross-entropy on positive samples, and a validity cross-entropy that separates correct from incorrect judgments (the model must not merely copy the input – it must judge). Sample counts and convergence are config-dependent: ~2000 samples / 3 epochs for the simple arithmetic suite; ~6000 samples / 8 epochs for the full logic-and-arithmetic suite (exp53 curve).

****Evaluation discipline.**** Judgment accuracy – full output-sequence reconstruction of the final truth token – is the only trusted metric; positional accuracy can mask structural errors. A single official evaluation path is used for every experiment (including ablations), because a divergence in evaluation branches silently invalidates comparisons. Position coverage is treated as an independent degree of freedom (a syntactic slot never seen in training does not generalize even with complete definitions). Diagnostics: per-token generalization (bimodal, §6), epoch curves, crash-culprit (which token most often breaks a judgment), and coverage audits. These make "definition completeness" and "relational sufficiency" checkable at the token level rather than as aggregate scores.

****Why inputs exclude answer traces.**** Supervision contains only judgment truth (correct/incorrect), never the step-by-step unfolding. If traces were given, the model could copy the unfolding (sequence cloning); without them it must induce the computational structure from definitions and compile it into weights. Learning is compilation. The compiled product is intuition: the reasoning fast path.

6 Experiments

All runs use the single official `run_exp` pipeline (config/metrics/model/samples/views archived). Judgment accuracy reported.

Experiment	Result (judgment acc)
Full suite, seed s1/s2 (9 logic gates + arithmetic)	1.000 / 1.000 (~6000 samples, 8 epochs)
Simple arithmetic suite (arith_multi_type)	1.000 (~2000 samples, 3 epochs)
imply with supervision (definition+position+judgment)	0.996
2000-digit addition extrapolation	1.000
Unseen radix [3..9] $\times$ 20 digits	OOD 0.998
Unseen domain / operator level (addition $\rightarrow$ multiplication)	generalizes; hyper-operator order coverage incomplete (see Limitations)
Joint Cartesian OOD (width $\times$ radix $\times$ nesting)	1.000 (exp80 joint)
Symbol permutation (exp41: 26080 digit renamings)	0.996–0.987
Three-channel consistency (exp50: 2000-digit carry chain)	1.000
Fast/slow path cost (exp51: 2000 digits)	construction 0.62s / intuition 0.086s (7 $\times$) / formal 0.069s (9 $\times$)
Convergence curve (exp53: epochs 1/2/3/5/8)	0.001/0.214/0.244/0.781/0.996
Step-skipping zero-decay (exp80: 2000-digit / radix 60 / 100% anonymized / joint)	1.000
Step-skipping machine form (exp90: 20-digit add)	construction 82 tokens/0.410ms $\rightarrow$ intuition 5 tokens/0.186ms (2.2 $\times$; honest lower bound, itoken outside vocab)

****Key experiments.****

- *Width*: training width $\leq L$, test 2000 digits – length-independence, refutes function-table memorization.
- *Radix*: base is an input parameter, not a structural constant – refutes decimal memorization.
- *Order/domain*: addition $\rightarrow$ multiplication level extrapolation succeeds; hyper-operator levels (power/root/tetration) are not fully covered because radix-base generation overflows for astronomical values – see Limitations. Iteration-chain structure is learned via A-layer arrows, not per-operator tables.
- *Symbol permutation (exp41)*: random permutation of 10 digit concept tokens (26080 renamings, structure preserved), tested on original tokens – invariance 0.996-0.987. The model learned structural relations (successor/iteration/carry), not digit semantics. Combined with exp80's 100% anonymization at 1.000, this is the strongest refutation of "mathematical knowledge as memory".
- *Three channels (exp50)*: construction = formal exactly (2/9/20/2000 digits); intuition 1.000 including a 2000-digit carry chain (999..9+1) – real state propagation, weights $\approx$ program.
- *Step-skipping zero decay (exp80)*: at fixed acquisition (8 epochs), extrapolation distance increases with no decay – 2000-digit (10 $\times$ width), radix 60 (6 $\times$), 100% digit anonymization, and joint width $\times$ radix all at 1.000. Acquisition and step-skipping generalization are decoupled.
- *Step-skipping machine form (exp90)*: the intuition path treats a 20-digit number as an atomic num token (5 tokens) versus the construction path's 82-token digit-wise unfolding. Honest caveat: itoken is outside the model vocab (pad id 0); the measurement is the sequence-length effect on forward cost (a lower bound). If the model were to truly learn itoken semantics, the intuition path would be an O(1) decision with a larger advantage.

****Per-token generalization: extreme bimodal distribution.**** The aggregate learning curve (exp53) is the union of per-token curves that are sharply bimodal:

- A learned concept reaches 100% by epoch 3 with no gradual transition;
- an unlearned concept stays at 0% and is *not* fixed by more epochs;
- intermediate states (30%, 50%) are essentially absent.

The apparent gradual rise of the aggregate curve is therefore the stacking of per-token "switch-ons": each concept is either compiled into weights (100%) or not (0%). This is the mechanism-level signature of the compilation account (C4). It also makes generalization failures attributable: an unlearned item points at the concept itself (its definition precision or its dependency/coverage), not at "insufficient training" – consistent with C1 and with the token-level diagnostics (per-token accuracy, crash-culprit, coverage audit) used throughout.

7 Semantic-Path Compilation

****Zero samples already execute.**** A new operator σ 's semantic closure is guaranteed by its definition chain: a base model that has never seen σ can still execute along the expansion path T_σ (construction channel) – correct, but long. Semantic closure does not depend on training.

****Few samples build the reasoning fast path.**** A small number of samples compiles a semantically equivalent intuition channel: $Sem_fast(\sigma) = Sem_expanded(T_\sigma)$ (verifiable, with fallback); $Cost_fast < Cost_expanded$. The intuition channel does not expand the semantic closure – it solves an execution-cost problem, not a new-semantics problem. Unlike classical distillation, the teacher and student are the *same model's* construction and intuition channels (construction-channel self-compilation), with a verifier and fallback.

****Terminology.**** Intuition is the *reasoning fast path*. It is hard to obtain (sufficient training; config-dependent epochs), but easy to generalize by skipping steps: it jumps from input to output over the intermediate construction steps. Speed is its nature as a reasoning path, not an independent optimization; step-skipping is the mechanism of its outward generalization. Honest caveat: the formal engine (eval_op) is faster in absolute terms (0.069s vs 0.086s at 2000 digits); the value of the intuition channel is end-to-end execution (no external oracle) and preserved step-skipping generalization.

8 Ablations

Ablation	Result	Supports
Degrees of freedom: factored (numeral_v4 correct) vs entangled/wrong-symmetry	entangling collapses generalization	G4/C1
Parallel-concept removal (exp20a/b: delete iff/xor or and/or supervision)	overall 0.798/0.814; deleted gate 0.000; **other gates 1.0-0.833** – the mutual-training chain (De Morgan duals / inverse arrows) breaks	G2
Symbol permutation (exp41)	0.996-0.987	G4
Three-channel consistency (exp50)	construction=formal=intuition (2000-digit carry chain, 1.000)	R/C5
Convergence curve (exp53)	monotone; full suite converges at 8 epochs – supports "intuition is hard to obtain"	G5
Negative-example coverage (neg 9-15)	neg 0.000-0.958	G2
Definition-chain token uncovered (value_three, 1 sample)	not learned; learned after coverage	G1
Definitionally impoverished (vocab 153)	requires 40 epochs	G0/G1
Wrapper-symbol mixing ("(")" vs "[]" same symbol)	training collapses	G4
Judgment vs positional (focused config)	positional 0.904 $\rightarrow$ judgment 0.000 (artifact)	metric discipline
Ordinary representation baseline (EXP-C1: radix-fixed / iteration-hidden / entangled)	pending	C1

****The ordinary-representation baseline (EXP-C1).**** The decisive control: same model, same data budget, but the representation is de-factored. We progressively destroy degree-of-freedom factorization and observe extrapolation collapse:

Control	Factorization destroyed	Result
C1d factored (exp02, reference)	none	zero-decay extrapolation (exp80: 1.000)
C1b iteration-hidden (done)	iterate structure not exposed (law power/root/tetration, iterate/inverse arrows, staircase removed)	overall 0.745 ($\Delta 0.25$); **root 0.000, tetration 0.000** – high-level operators collapse; **power 1.000** (derivable from multiplication: $2^3 = 2 \times 2 \times 2$); addition/multiplication 1.000
C1a radix-fixed (re-designing)	radix made a structural constant (train base-10 only, no base/cardinality parameter)	pending – expected: unseen radix (base 16/60) collapses; first attempt (strip base tokens) over-destroyed the numeral representation (0.003), being re-designed to keep base-10 computable while removing the radix degree of freedom

| Clc entangled (pending) | composite concepts merged into single tokens (degrees of freedom fused) | pending – expected: new combinations collapse |

****Clb analysis.**** The iteration-hidden result is precise evidence for the definition-chain account (G1): collapsing operators are exactly those whose definition chains genuinely depend on the iteration structure (root = inverse of power-iteration; tetration = power-iteration at the next level), while power survives because it is reducible from multiplication. Factorization of the iteration degree of freedom is thus the **cause** of order extrapolation – not for all operators uniformly, but exactly for those whose definitions require it. This is a sharper claim than “factorization helps generally.”

****Methodological discipline (from a withdrawn experiment).**** An earlier zero-supervision implication experiment (EXP-10, 0-IMPLY) was fully withdrawn: its reported “semantic emergence” (bare truth table 0.75 / deep nesting 0.938) was an artifact of last-token evaluation plus syntactic misplacement (an evaluation-branch inconsistency that made all variants produce identical results). Under the official full-sequence judge, implication scored 0.000; root cause: positional misplacement plus insufficient sample count. We record the lessons – full-sequence reconstruction is the only trusted metric; evaluation must use the single official pipeline; position coverage is an independent degree of freedom – and the correct path for implication is the triple definition+position+judgment supervision (0.996). The withdrawn experiment contributes methodological discipline, not evidence.

9 Related Work

- ** λ -calculus and formal-system families.**** Interpreted object, not benchmark: primitives pre-supposed (variable/abstraction/application); Church encoding erases structure. The “neural λ -calculus” program (Flach, Moreira & Lamb 2023) exemplifies the λ -tokenize-then-train route – a seq2seq Transformer trained on de Bruijn-indexed λ -syntax to perform β -reduction, with generalization confined to the training distribution. The structural reading of iteration – that iteration is composition ($f^{m+n} = f^m \circ f^n$), that the Church zero is not a base, and that category theory turns iteration into a universal property – is developed in detail in our internal study (λ -base iteration $\rightarrow$ category theory); standard foundations: Lambek 1968 (fixpoint theorem for complete categories), Goguen et al. 1977 (initial algebra semantics), Martin-Löf 1984 (intuitionistic type theory), Fiore-Plotkin-Turi 1999 (abstract syntax and variable binding).
- **Definition theory and definition forms.**** Five definition forms (explicit / inductive / recursive / axiomatic / implicit) follow Gupta 2023 (SEP “Definitions”) and Belnap 1993 (“On Rigorous Definitions”); inductive definitions per Aczel 1977; implicit definitions require conservativeness and eliminability (Suppes 1957; Gupta-Belnap 1993 revision theory); definitional machinery in proof assistants (Lean, Rocq, Isabelle). No single prior system unifies concept definitions and grammar definitions in one schema – this is a gap this work fills.
- **Semiotics and signifier/signified separation.**** Saussure 1916 (arbitrariness, differential principle); Peirce’s sign triad; Frege 1892 (Sinn/Bedeutung); Harnad 1990 (symbol grounding). The maintenance logic of many-to-many symbol $\rightarrow$ concept mappings follows WordNet (Miller 1990; Fellbaum 1998), Word Sense Disambiguation (Navigli 2009), and SKOS (Miles & Bechhofer 2009). Our S-layer operationalizes ambiguity as first-class and context-convergent, as argued in our symbol $\rightarrow$ concept mapping survey.
- **Syntax as a definable object and grammar engineering.**** W-grammars (van Wijngaarden 1965; ALGOL 68 1975); parsing expression grammars (Ford 2004); invertible syntax descriptions unifying parsing and pretty-printing (Rendel & Ostermann 2010); concrete syntax as typed code with explicit decoders (our survey of syntax-as-definable-object). This motivates the native-grammar design: grammar itself is tokens.
- **Variable binding and scope.**** de Bruijn 1972 (nameless dummies); higher-order abstract syntax (Pfenning & Elliott 1988; Harper-Honsell-Plotkin 1993 LF; McDowell & Miller 2002); nominal logic (Gabbay & Pitts 2002); locally nameless (Charguéraud 2012). Binding arity and scope verification (our G-layer) follow these; position/scope coverage is an independent degree of freedom.
- **Systematic generalization.**** Lake et al. 2017; Keyser et al. 2020 (COGS); SCAN. These document the phenomenon; this work provides the genesis (why and when generalization holds) as formalized induction.
- **Neural-symbolic arithmetic modules.**** NALU/NAU (fixed operator sets); this work uses a unified parameter set with zero-sample semantic closure for new operators.
- **Length generalization.**** Position-encoding and recurrent-elaboration engineering under fixed representations; this work removes length-dependence at the representation layer.
- **Representation learning.**** Disentangled/compositional structured representations study what representations look like; this work studies how representation determines learnability, treating representation design as a contribution.
- **Program-to-neural compilation.**** Learning to Compile Programs to Neural Networks (ICML 2024); inference compilation; shortcut distillation. Differences: same-model construction-channel self-compilation, no semantic-closure expansion, verifier + fallback.
- **Dual-process cognition.**** Kahneman (System 1/2) grounds G5: intuition as the reasoning fast path.
- **Philosophy of mathematics – three-channel equivalence.**** The mathematical structures are all prior art (de Bruijn 1972; combinatory logic; Bourbaki structuralism; category theory; Mizar/Lean/Isabelle) and are explicitly not claimed. The claim is only three-channel equivalence – construction/intuition/formal rewriting mutually compilable within one system – the neural implementation of Computational Trinitarianism (Curry-Howard: programs = proofs = types), equivalence between execution channels rather than between logics. Stance: anti-Platonist – intuition is compiled construction, not innate.
- **Anti-nativism.**** Fodor (language of thought) and Chomsky (universal grammar) are the opposing positions; empirical counterevidence (definitionally-impoorished tokens do not generalize) is provided.

10 Limitations

- **Serialization (positional bias).**** Position coverage is an independent degree of freedom: a syntactic slot never covered in training does not generalize even with complete definitions. Positional dependence is not fully removable in autoregressive generation.
- **Finite resources.**** Weight/sample/epoch bounds exist; convergence is config-dependent (3 simple, 8 full – exp53); hyper-operator results can be astronomically large, so judgment negatives use small domains with overflow guards.
- **Operator-order coverage is incomplete.**** Hyper-operator levels (power/root/tetration) beyond the trained level are not fully evaluated: radix-base generation overflows for astronomical results, so only addition $\rightarrow$ multiplication level extrapolation is measured. This is a coverage limitation, not a negative result. Likewise, symbol permutation (exp41) permuted digit concepts only; full operator-symbol permutation is pending.
- **Supported calculus $\neq$ unrestricted mathematics.**** Only structures with formalized definitions (within the five definition forms) are covered; no claim of general mathematics or autonomous concept discovery; new concepts still require manual definition registration.
- **Value space/representation.**** Currently boolean judgments and counting structures; the full value space (

fractions/complex) is partially connected; the numeral-value side is not fully open.

6. ****Intuition is statistical, not guaranteed.**** The intuition channel performs structural location statistically; semantic equivalence requires a verifier/fallback (construction channel) as backing.

11 Conclusion

- Generalization is the formalization of induction (§3): definition completeness (G1) plus relational sufficiency (G2) yields systematic generalization in a minimal model – engineered, reproducible. Its per-concept learning curves are bimodal (compiled vs not), making failures attributable to definitions and coverage, not to training volume.
- Representation is the contribution: factored degrees of freedom turn generalization into a low-complexity learning problem; generalization is written into the geometry of the representation, not left to the model to discover. The de-factoring control (EXP-Clb) confirms the causal role: hiding the iteration structure collapses exactly those operators whose definitions depend on it (root, tetration), while reducible ones (power) survive.
- Weight compilation: training compiles construction into intuition; supervision excludes traces, so structure is learned. Intuition is the reasoning fast path (G5) – hard to obtain, easy to generalize by skipping steps, with zero decay across width, radix, and anonymization.
- Three-channel equivalence (R): construction/intuition/formal rewriting are mutually compilable in neural implementation (exp50: 2000-digit carry chain, all three channels 1.000) – the engineering unification of a century-old split in the foundations of mathematics.
- Future: larger domains; autonomous concept formation (inducing new definitions from data); cross-modality (extensions of symbol permutation).

Appendix

- A. Full experiment tables: config/seed/judgment per-token shortfalls/conclusions (docs/paper_data/results.csv).
- B. Grammar: complete gtoken/ptoken listing (data-driven source); presentation examples (add_infix, and_infix, not_prefix, succ_prefix).
- C. More examples: training sample sequences (numeral_split, law, three-channel inputs, judgment sequences).
- D. Training configs: complete JSON (exp02 suite: synth.samples composition + judge + train).
- E. Lean proofs: definition-chain completeness (no dangling references, cycle certificates) and the compilation correctness of three-channel equivalence – discipline verification, not a new mathematical claim.

References

[Bibliography in progress; citations sourced from the project's cross-disciplinary surveys (docs/□□□□/). Verified full records to be completed.]

- Aczel, P. 1977. An Introduction to Inductive Definitions. Handbook of Mathematical Logic.
- Adámek, J., Herrlich, H., Strecker, G. 2004. Abstract and Concrete Categories.
- Barendregt, H. 1984. The Lambda Calculus: Its Syntax and Semantics.
- Belnap, N. 1993. On Rigorous Definitions.
- Chaguéraud, A. 2012. The Locally Nameless Representation. JAR.
- de Bruijn, N. G. 1972. Lambda Calculus Notation with Nameless Dummies.
- Fiore, M., Plotkin, G., Turi, D. 1999. Abstract Syntax and Variable Binding. LICS.
- Flach, J. M., Moreira, Á. F., Lamb, L. C. 2023. Towards a Neural Lambda Calculus: Neurosymbolic AI Applied to the Foundations of Functional Programming. arXiv:2304.09276.
- Fodor, J. 1975. The Language of Thought.
- Ford, B. 2004. Parsing Expression Grammars. POPL.
- Frege, G. 1892. Über Sinn und Bedeutung.
- Gabbay, M. J., Pitts, A. M. 2002. A New Approach to Abstract Syntax with Variable Binding. FAC.
- Goguen, J., Thatcher, J., Wagner, E., Wright, J. 1977. Initial Algebra Semantics and Continuous Algebras. JACM.
- Gupta, A. 2023. Definitions. Stanford Encyclopedia of Philosophy.
- Gupta, A., Belnap, N. 1993. The Revision Theory of Truth.
- Harper, R., Honsell, F., Plotkin, G. 1993. A Framework for Defining Logics. JACM.
- Harnad, S. 1990. The Symbol Grounding Problem. Physica D.
- Kahneman, D. 2011. Thinking, Fast and Slow.
- Keyzers, D. et al. 2020. Measuring Compositional Generalization: A Comprehensive Method. ICLR.
- Lake, B., Ullman, T., Tenenbaum, J., Gershman, S. 2017. Building Machines That Learn and Think Like People. BBS.
- Lambek, J. 1968. A Fixpoint Theorem for Complete Categories. Math. Z.
- Mac Lane, S. 1971. Categories for the Working Mathematician.
- Martin-Löf, P. 1984. Intuitionistic Type Theory. Bibliopolis.
- McDowell, R., Miller, D. 2002. Reasoning with Higher-Order Abstract Syntax in a Logical Framework. ACM TOCL.
- Miles, A., Bechhofer, S. 2009. SKOS Reference. W3C.
- Miller, G. et al. 1990. Introduction to WordNet.
- Navigli, R. 2009. Word Sense Disambiguation: A Survey. ACM CSUR.
- Pfenning, F., Elliott, C. 1988. Higher-Order Abstract Syntax. PLDI.
- Rendel, T., Ostermann, K. 2010. Invertible Syntax Descriptions. Haskell Symposium.
- Saussure, F. de. 1916. Cours de linguistique générale.
- Suppes, P. 1957. Introduction to Logic.
- van Wijngaarden, A. 1965. Orthogonal Design and Description of a Formal Language.
- [Additional survey-derived records: Burris & Sankappanavar 1981 (Universal Algebra); Kripke 1963 (Modal Logic); Hamblin 1973 (Questions); Groenendijk & Stokhof 1984; Kamp 1981 (DRT); Ogden & Richards 1923 (The Meaning of Meaning); Cruse 2000; Kilgariff 1997; Cajori 1928 (A History of Mathematical Notations); Putnam 1975 (The Meaning of "Meaning"); Rosch 1973 (Natural Categories); Enderton 2001; Hodges 1993 (Model Theory); Gentzen 1934/35; Prawitz 1965; Russell 1905 (On Denoting); Li & Vitányi 2008 (Kolmogorov Complexity); Fiore & Szamozvancev 2022; Cray & Harper 2006.]